

Local service provider

Table of Contents

Chapter - I

Introduction

1.1	Project Overview	1
1.2	Objectives	2
1.3	Purpose & Relevance	3
1.4	Problem Statement	4
1.5	Scope of the Project	5
1.6	Key Features	6
1.7	Project Category	8

Chapter – II

Tools and Requirements

2.1	Tools & Technologies Used	9
2.2	Hardware Requirements	10
2.3	Software Requirements	10

Chapter – II

Problem Definition and Project Planning

3.1	Problem Definition	11
3.2	Functional Requirements	12
3.3	Non-Functional Requirements	13
3.4	Project Phases	15
3.5	Use Case Descriptions	17

Chapter – III

Solution Scope

4.1	What It Solves	18
4.2	Key Users and Their Benefits	18
4.3	Value Over Existing Methods	19

Chapter-V

System Analysis and Design

5.1	Used Software Development Life Cycle (SDLC) Model	20
5.2	Feasibility Study	22
5.3	ER Diagram	23
5.4	Relational Database Design	25
5.5	Data Flow Diagrams	26
5.6	Database Design	30
5.7	Data Dictionary	31

Chapter-VI

Project Structure

6.1	Modules	38
6.2	Data Structures and APIs	39
6.3	Process Logic / Workflows	39
6.4	Screens and Their Functions	40

Chapter-VII

Testing

7.1	Unit Testing	41
7.2	Functional Testing	43
7.3	Tools Used for Testing	44

Chapter-VIII

Limitations and Future Enhancements

8.1	System Limitations	45
8.2	Future Enhancements	46

	Conclusion	47
--	------------	----

	User guide	49
--	------------	----

Chapter - I

Introduction

1.1 Project Overview

"Local Service Provider" is an innovative, full-stack web application designed to connect users with local service providers in a simplified, efficient, and user-friendly manner. Developed using the Express, React and with PostgreSQL for enhanced relational data handling—the project aims to solve the fragmented nature of local service discovery by providing a centralized platform for booking, managing, and reviewing various services.

The application serves two primary user groups: service seekers (end-users) and service providers (businesses or individuals offering local services). End-users can register, log in, search for services, add items to a wishlist, and make service bookings, while service providers can list their offerings through administrative tools. A powerful admin dashboard further enables system oversight, user verification, service moderation, and analytics, ensuring the integrity and smooth operation of the platform.

The platform is built with responsiveness, security, and scalability in mind. The use of React.js ensures a dynamic user experience, while Express.js and Node.js provide a robust backend infrastructure. PostgreSQL handles data relationships efficiently, offering

structured queries and relational consistency that enhance both performance and maintainability.

The system also addresses common issues in traditional service-finding methods, such as lack of trust, unverified information, and inefficiency in communication. With features like search filters, secure login via JWT, password resets, and a dedicated wishlist and booking section, "Local Service Provider" is positioned as a smart solution for community-level service engagement.

This application is ideal for use in small towns, urban neighborhoods, or even campus environments where the demand for reliable local services is consistent and underserved. It not only brings digital access to everyday services but also empowers local providers with visibility and reach.

1.2 Objectives

The objectives of the "**Local Service Provider**" project are designed to comprehensively address the current gaps in local service accessibility, user trust, and digital integration. By leveraging modern web technologies and intuitive design, the project aims to build a robust and sustainable service ecosystem. The core objectives include:

- **Digital Integration of Local Services:** To create a unified online portal where users can browse, book, and manage a variety of local services.
- **Enhanced User Convenience:** To reduce the effort, time, and uncertainty involved in finding reliable service providers through traditional means.
- **Platform for Service Provider Growth:** To offer small and independent service providers, a digital presence that improves their visibility and accessibility to potential clients.
- **Trust and Transparency:** To implement verification protocols, user ratings, and booking histories that establish accountability and reliability on both sides.
- **Secure and Scalable System Architecture:** To develop a highly secure system using JWT for authentication, bcrypt for password protection, and PostgreSQL for consistent data integrity.

- **Feature-rich Interface:** To design an application with modern UX/UI components, including booking history, wishlist, admin panel, password reset, and search functionalities.
- **Administrative Oversight and Control:** To equip admins with tools to oversee platform activity, moderate listings, manage user roles, and generate reports.
- **Extensibility and Future-Proofing:** To ensure the application architecture supports future enhancements such as real-time chat, integrated payments, and geolocation-based filtering.

These objectives collectively focus on usability, security, efficiency, and adaptability, ensuring the application addresses both immediate and future needs of its users.

1.3 Purpose & Relevance

The primary purpose of the "**Local Service Provider**" is to modernize the way individuals interact with local service providers by creating a digital bridge between them. This platform is intended to replace outdated, disorganized, and often unreliable systems of service discovery with a streamlined, transparent, and highly accessible digital infrastructure.

The system aspires to serve as a one-stop destination where users can seamlessly find, book, and manage essential services ranging from household maintenance to tutoring and personal care. For service providers, it offers an opportunity to showcase their offerings, reach new customers, and manage client relationships more effectively—all without the need for expensive marketing or middlemen.

In terms of relevance, this project directly addresses several societal and technological trends:

- **Digital Transformation:** With the growing penetration of internet access and smartphones, consumers now expect services to be discoverable and accessible online. This project aligns with global movements toward digital convenience.
- **Economic Inclusion:** It provides micro and small business owners, often operating informally, a chance to establish a digital footprint and expand their customer base.
- **Trust and Community Empowerment:** By incorporating reviews, verified listings, and booking history, the platform builds a community of trusted service exchanges, reducing the uncertainty typically associated with informal arrangements.

- **Scalability and Adaptability:** While the system begins at a local or community level, its architecture is capable of expanding to support city-wide or even regional service operations.
- **Post-Pandemic Relevance:** With the shift toward contactless service arrangements and remote planning, such platforms have become more essential than ever before.

Overall, the project is highly relevant in today's fast-evolving digital landscape and addresses both immediate and long-term needs of a community-driven service ecosystem.

1.4 Problem Statement

In today's fast-paced digital world, accessing basic home and personal services should be simple, yet many communities continue to face significant challenges when searching for local service providers. Most individuals still rely on outdated and informal methods—such as neighborhood referrals, word-of-mouth, social media groups, or flyers posted in public spaces—which often lack transparency, reliability, and efficiency.

The core problems faced by users in the current system include:

- **Fragmented Access:** There is no centralized, structured platform where users can browse and compare different service providers based on verified information or reviews.
- **Trust Deficit:** Without a feedback or rating mechanism, users are unable to gauge the reliability and performance of a provider before making a booking.
- **Limited Reach for Providers:** Local professionals, particularly small-scale service providers, struggle to gain visibility due to limited or no digital presence, missing out on potential clients.
- **Security and Privacy Concerns:** Informal communication and unregulated platforms often compromise user privacy, with personal contact details exchanged freely without control or oversight.
- **Lack of Transaction History:** Without proper documentation of service history, it becomes difficult for users to track previous orders, manage repeat services, or resolve disputes.

These issues are particularly pressing in semi-urban and urban settings where the demand for home-based services is high, but access remains disorganized and manual. The "Local

Service Provider" project aims to tackle these challenges by creating a digital platform that streamlines service discovery, fosters trust through transparent interactions, and promotes equitable access for service providers. The result is a system that improves user convenience, expands professional visibility, and brings much-needed organization to the local service economy.

1.5 Scope of the Project

The "**Local Service Provider**" project offers a wide and scalable scope aimed at organizing and optimizing how local services are discovered, booked, and managed. It is designed to be a flexible and extensible platform that addresses the needs of both consumers and service providers within a community. The project scope includes, but is not limited to, the following key areas:

- **User Registration and Profile Management:**
 - Secure account creation and login system for users and admins.
 - Profile dashboard for users to manage personal data, bookings, and saved services.
- **Service Listing and Categorization:**
 - Admin can define, update, and manage service categories.
 - Users can browse or search for services based on category, name, or keywords.
- **Booking and Wishlist Features:**
 - Users can book services with specified date, time, and location details.
 - Ability to add services to a personal wishlist for future reference.
- **Admin Control Panel:**
 - Full administrative access to monitor users, services, and activity logs.
 - Approve, reject, or edit service listings and user accounts.
- **Search and Filter Capabilities:**
 - Real-time, responsive search bar for quick access to services.
 - Filters by category, popularity, or availability.
- **Security and Session Management:**

- Encrypted password storage using bcrypt.
- JWT-based session control for secure logins and access management.
- **Database Integration with PostgreSQL:**
 - Robust relational data modeling for structured service, user, and booking data.
 - Optimized queries for performance and integrity.
- **Responsive Frontend Interface:**
 - Developed using React.js for smooth UI/UX.
 - Accessible on desktop, tablet, and mobile devices.
- **Scalability and Extensibility:**
 - Designed to accommodate additional features like payment integration, geolocation filters, and service provider profiles.

This scope ensures the system not only meets the current requirements of a local community-based service platform but is also well-equipped to scale as demand and complexity grow.

1.6 Key Features

The "**Local Service Provider**" is enriched with a comprehensive set of features tailored to meet the dynamic needs of modern users and administrators alike. These features have been designed to ensure maximum usability, efficiency, and future scalability:

- **Robust User Authentication:**
 - Secure login and registration using JWT (JSON Web Tokens) and bcrypt encryption.
 - Session management and role-based access control for users and admins.
- **Dynamic Service Listings:**
 - Users can view detailed listings categorized by service types (e.g., plumbing, electrical, tuition).
 - Each listing includes service descriptions, provider credentials, and optional images.

- **Advanced Search and Filters:**
 - A real-time search bar enables users to find services quickly.
 - Filtering options by category, availability, popularity, or keyword.
- **Interactive Booking System:**
 - Users can book services by selecting a time, date, and location.
 - Confirmation system with order status updates (e.g., pending, accepted, completed).
- **Wishlist and Booking Management:**
 - Users can bookmark preferred services for future reference.
 - Track, modify, or cancel bookings from a personalized dashboard.
- **Admin Control Panel:**
 - Admins can manage all user accounts and monitor service provider activities.
 - Tools for reviewing, approving, editing, or deleting service listings.
- **Secure Password Reset:**
 - Password reset flow via email verification or security question validation.
 - Prevents unauthorized access with time-limited reset tokens.
- **Mobile-Responsive UI:**
 - Developed using React.js with responsive design principles.
 - Optimized for accessibility across desktop, tablet, and mobile devices.
- **PostgreSQL-Based Data Integrity:**
 - Utilizes relational schemas for handling complex user-service relationships.
 - Ensures reliable data storage and high-performance querying.
- **Scalability & Future Integrations:**
 - Built on modular architecture to support features like payment gateways, GPS-based filtering, reviews/ratings, and real-time messaging.

These features collectively create a modern, reliable, and scalable platform for both everyday users and service providers, ensuring an all-around productive experience.

1.7 Project Category

The "**Local Service Provider**" project falls under the following categories based on its technology stack, data handling approach, and system design characteristics:

This project is a comprehensive example of full stack development as it includes both frontend and backend components. It uses React.js for the client-side interface and Node.js with Express.js for the backend server logic. The interaction between the frontend and backend is facilitated using RESTful APIs.

Unlike many MERN, this project uses PostgreSQL—a relational database system. The use of PostgreSQL allows structured data storage and efficient relationship handling among entities like users, services, bookings, and admins, which is ideal for a platform requiring integrity and complex data queries.

The core functionality of the platform revolves around Create, Read, Update, and Delete operations:

- Users create accounts, book services, and manage wishlists.
- Admins add/edit service listings, monitor user data, and manage system operations.
- Bookings and listings are updated and deleted as needed, ensuring data remains relevant and accurate.

Security is integral to this system. The platform incorporates multiple layers of security:

- JWT-based authentication for session management.
- Password hashing with bcrypt for credential safety.
- Role-based access control to ensure that users and admins have different permission levels.

Chapter - II

Tools and Requirements

2.1 Tools & Technologies Used

The development of "**Local Service Provider**" utilized a range of modern technologies that together support a scalable, responsive, and secure web application. These tools and frameworks are as follows:

- **Frontend:**
 - **React.js:** Used for building the interactive user interface with component-based architecture.
 - **Material UI / Custom CSS:** For styling the user interface with utility-first classes and mobile responsiveness.
 - **Axios:** To facilitate HTTP communication between frontend and backend.
- **Backend:**
 - **Node.js:** Provides the JavaScript runtime environment for server-side scripting.
 - **Express.js:** Framework built on top of Node.js used for building RESTful APIs.
- **Database:**
 - **PostgreSQL:** A powerful open-source RDBMS used for managing structured data with relationships.

- **Security:**
 - **bcrypt.js:** For encrypting user passwords.
 - **jsonwebtoken (JWT):** For generating secure tokens for authentication.
- **Development Tools:**
 - **Visual Studio Code:** Primary code editor.
 - **Postman:** For testing API endpoints.

2.2 Hardware Requirements

To develop, test, and deploy the "**Local Service Provider**" application, the following hardware configurations are recommended:

- **Minimum Requirements (Development/Testing):**
 - Processor: Intel Core i3 (or equivalent)
 - RAM: 4 GB
 - Storage: 2 GB of free space
 - Display: 1366x768 resolution

2.3 Software Requirements

- **Operating System:**
 - Windows 10/11, macOS, or any modern Linux distribution
- **Browser:**
 - Google Chrome, Mozilla Firefox, Microsoft Edge (latest versions recommended)
- **Runtime & Libraries:**
 - Node.js (v16 or higher)
 - npm (Node Package Manager)
- **Database Engine:**
 - PostgreSQL (version 13 or higher)

These tools and system requirements collectively support the development and execution of a high-performance web application tailored for local service management.

Chapter -III

Problem Definition and Project Planning

3.1 Problem Definition

Accessing trusted, reliable, and nearby service providers continues to be a persistent issue in many communities, especially where digital adoption is minimal or fragmented. Traditional methods such as word-of-mouth referrals, community boards, or social media groups are often inefficient, unstructured, and offer no assurance of credibility. These systems not only lack transparency but also make it difficult for new service providers to gain visibility or for users to make informed choices.

The absence of a centralized platform for local services results in several key challenges:

- **Disorganization and Fragmentation:** Users must rely on multiple sources, often inconsistent and outdated, to find help for daily service needs.
- **Lack of Accountability:** Without a formal system, there is no mechanism for reviews, ratings, or verification—leading to trust issues.
- **Inaccessible Service Providers:** Local professionals, especially new or small-scale providers, lack a proper digital platform to advertise their services.

- **Data Loss and Manual Processes:** Offline interactions leave no trail or history, making it difficult to manage or revisit bookings.
- **Privacy and Security Risks:** Sharing personal details on public or informal platforms can expose users to spam or misuse of information.

The "Local Service Provider" is designed to directly address these limitations by offering a secure, organized, and user-friendly web application. It provides a platform for users to discover, book, and manage services while empowering service providers to increase their reach through digital tools. The system fosters a more reliable and efficient service environment for all stakeholders involved.

3.2 Functional Requirements

The following functional requirements define the core system capabilities that ensure users and administrators can effectively interact with the "**Local Service Provider**" application. Each feature is designed to deliver a smooth, reliable, and secure user experience:

- **User Registration and Login:**
 - Users must be able to register by providing essential information such as name, email, and password.
 - A secure login system using JWT tokens should be implemented for session management.
- **Role-Based Access Control:**
 - System must differentiate between regular users and admin accounts.
 - Admin privileges should include access to content moderation and system configuration.
- **Service Listings:**
 - Users must be able to view services organized under various categories.
 - Each listing should display service name, description, contact info (if permitted), and provider identity.
- **Search and Filter Services:**
 - Real-time search functionality should allow users to quickly locate services.
 - Filters should be available based on category, date availability, popularity, or provider name.

- **Booking Management System:**
 - Users should be able to book a service by specifying details like preferred date, time, and address.
 - A booking confirmation and status tracking feature (e.g., pending, accepted, completed) should be available.
- **Wishlist Module:**
 - Users can save or bookmark services for future use.
 - Wishlist items should be displayed in the user dashboard.
- **Admin Panel for Monitoring:**
 - Admin can view and manage all users and service listings.
 - Admins should be able to approve, reject, or remove listings and flag/report users.
- **Password Recovery:**
 - A password reset mechanism using security questions or email token verification must be in place.
- **Booking History and User Dashboard:**
 - Each user should have access to a personal dashboard displaying booking history and current status.
- **Notifications and Alerts:**
 - Users and admins should receive relevant status updates, confirmations, or alerts based on actions performed in the system.

These functional requirements form the operational backbone of the platform, ensuring that all core tasks—from user registration to service management—are efficiently handled.

3.3 Non-Functional Requirements

The non-functional requirements define the quality attributes of the "**Local Service Provider**" application. These requirements focus on how the system performs and behaves under various conditions, ensuring long-term maintainability, security, usability, and performance.

- **Security:**
 - All sensitive data, including passwords, must be encrypted using industry-standard algorithms like bcrypt.
 - Authentication and authorization should be implemented using JWT tokens with role-based access control.
 - All API endpoints must validate user sessions to prevent unauthorized access.
- **Performance:**
 - The system should handle multiple user requests simultaneously with minimal response time.
 - Search queries and data retrieval must be optimized for performance using PostgreSQL indexes and relational joins.
- **Scalability:**
 - The platform should support future growth in terms of user base, number of services, and modules like payments and messaging.
 - Codebase should be modular to allow easy integration of new features without disrupting existing functionality.
- **Availability and Reliability:**
 - The system should be consistently available with minimal downtime.
 - Proper error handling and logging should be implemented to catch, report, and fix issues efficiently.
- **Usability and Accessibility:**
 - User interfaces should be intuitive, clean, and responsive across devices (mobile, tablet, desktop).
 - Navigation should be simple, and elements like forms, buttons, and messages should follow UI/UX best practices.
- **Maintainability:**
 - Code should follow standard conventions and be properly documented to support ongoing development.
 - Version control with Git should be used for tracking changes and collaboration.

- **Data Integrity and Consistency:**
 - PostgreSQL constraints such as primary keys, foreign keys, and unique indexes should be applied where applicable.
- **Responsiveness:**
 - All pages must load quickly and adapt to different screen sizes using responsive design techniques.

These non-functional requirements ensure that the system not only works correctly but also performs efficiently and reliably in a real-world environment.

3.4 Project Phases

To successfully develop and deploy the "**Local Service Provider**" platform, the project was structured into well-defined phases. These phases ensured a clear progression from ideation to implementation, while also allowing flexibility for iteration and enhancement:

1. **Requirement Analysis and Planning:**
 - a. Conducted stakeholder discussions to understand key requirements.
 - b. Identified target users, service provider roles, and admin responsibilities.
 - c. Defined the scope, goals, deliverables, and timeline using tools like Gantt charts.
2. **System Design and Architecture:**
 - a. Developed wireframes for major interfaces and screens.
 - b. Designed database schema using PostgreSQL with ER diagrams.
 - c. Defined application architecture including API structure, backend logic, and frontend interactions.
3. **Frontend and Backend Development:**
 - a. Created reusable React.js components for UI, integrated responsive design principles.

- b. Implemented Express.js APIs to handle service requests, bookings, user roles, and authentication.
- c. Set up PostgreSQL for managing structured data relationships securely and efficiently.

4. Integration and Testing:

- a. Integrated frontend and backend components to ensure seamless user experiences.
- b. Performed unit testing, API testing (with Postman), and interface testing.
- c. Identified and resolved bugs, UI glitches, and database anomalies.

5. Deployment:

- a. Deployed the application to a local server or a cloud environment.
- b. Set up runtime environments, configured environment variables, and monitored system logs.
- c. Ensured database migrations and static asset management were properly handled.

6. Documentation and User Guide:

- a. Prepared detailed documentation for user workflows, admin functions, and API endpoints.
- b. Created guides for new users and administrators to understand system functionality.

7. Maintenance and Enhancement:

- a. Gathered feedback from test users.
- b. Identified potential areas for enhancement (e.g., payment integration, chat systems).
- c. Scheduled regular updates for feature improvements and performance optimization.

This phased approach allowed for effective resource utilization, continuous evaluation, and timely delivery of a reliable product.

3.5 Use Case Descriptions

Use Case	Description
User Registration	A user creates a new account by providing essential details. Passwords are securely hashed, and a confirmation message is sent upon successful registration.
User Login	A registered user logs in using valid credentials. A JWT token is generated for authenticated session management.
Admin Verification	Admin verifies new users or manages service listings and user activities to ensure platform integrity and content accuracy.
Service Listing	Verified users or admins can add a new service with details such as name, category, description, contact, and availability.
Browse Services	Logged-in users can browse services with sorting/filtering based on categories, popularity, or Services.
Service Booking	A user selects a service, provides date, time, and address, and confirms the booking request. Booking status is tracked.
Wishlist Feature	Users can save preferred services to their personal wishlist for future reference.
Booking Management	Users can view, edit, or cancel their service bookings from their dashboard.
Admin Moderation	Admins can approve, reject, or delete user accounts, service listings, and monitor platform statistics.
Leave Feedback	After completion of service, users may provide feedback or ratings to help future customers.
Profile Management	Users can view and update their personal information and manage their service interactions.
Password Reset	Users can reset passwords securely through email/token-based authentication or security questions.

Chapter - IV

Solution Scope

4.1 What It Solves

The "**Local Service Provider**" offers a direct solution to the disorganized and unreliable systems currently used to find local services. It centralizes access to trusted service providers, streamlines the booking process, and fosters transparent interactions between users and providers. By eliminating manual and fragmented methods, the platform:

- Enables users to easily discover verified and reviewed local service providers.
- Reduces dependency on referrals and social media by offering real-time search and booking.
- Protects user data with secure login and encrypted session management.
- Empowers providers by giving them visibility and direct access to customers.
- Enhances trust through booking histories, structured profiles, and feedback mechanisms.

4.2 Key Users and Their Benefits

End Users (Customers)

- Access to a variety of local services from a single, easy-to-use platform.

- Ability to search, filter, and compare providers based on categories or preferences.
- Save time by booking services directly with minimal hassle.
- Maintain history of bookings and manage appointments efficiently.
- Benefit from secure password recovery and personalized dashboards.

Service Providers

- List their services and gain online visibility without the need for external advertising.
- Reach a broader local audience beyond traditional word-of-mouth channels.
- Receive and manage service bookings through an intuitive interface.
- Improve reputation through client ratings and reviews.

Administrators

- Control over platform content and user management.
- Monitor platform activity and ensure service quality.
- Resolve conflicts and maintain system integrity through moderation tools.

4.3 Value Over Existing Methods

Compared to traditional and informal service discovery methods, "**Local Service Provider**" provides significant advantages:

- **Centralization:** A single platform replaces multiple disconnected sources.
- **Verification:** User and provider authentication increases credibility.
- **Structure:** Bookings, feedback, and search functionality are standardized.
- **User Empowerment:** Transparent choices based on data rather than blind trust.
- **Efficiency:** Reduces time spent searching for services and simplifies communication.
- **Scalability:** Designed to adapt and grow, supporting advanced features in the future.

Chapter-V

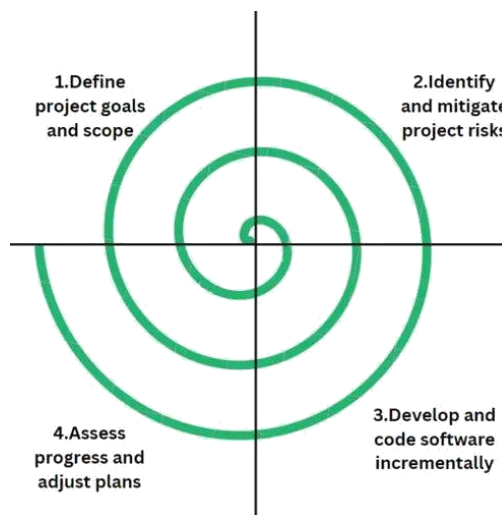
System Analysis

5.1 Used Software Development Life Cycle (SDLC) Model

For the development of the **Local Service Provider**, the **Spiral Model** of SDLC was adopted. This model combines elements of both **iterative** and **waterfall** models with a strong emphasis on **risk analysis**.

Each cycle in the Spiral Model consists of four key phases:

- **Planning:** Defined objectives and gathered requirements based on the challenges faced by students in second-hand book exchanges.
- **Risk Analysis:** Identified issues such as security, privacy, and user verification; addressed using JWT authentication, bcrypt encryption, and admin approval mechanisms.
- **Engineering:** Developed using React JS and MUI (frontend), Node.js and Express (backend), and MySQL (database). Modules were incrementally built and tested.
- **Evaluation:** Regular reviews and testing were conducted, and feedback was incorporated into subsequent iterations for continuous improvement.



Why Spiral Model?

- Allows incremental development and frequent revisions.
- Focuses on identifying and mitigating risks early.
- Supports changes during different phases of the project.

Phases Followed:

1. Planning Phase: Requirements Gathering and Feasibility Analysis

We defined the scope of the platform—connecting users with local service providers. Key features included user registration, service listings, search filters, bookings, and reviews. We also assessed the technical feasibility of using PostgreSQL with MERN, confirming that it aligned with our timeline and resources.

2. Risk Analysis: Identification of Risks and Mitigation Strategies

This phase involves identifying risks such as technical limitations, team skill gaps, or changing requirements. Each risk is assessed for impact and likelihood, with mitigation strategies like backup plans and buffer time. Ongoing monitoring helps the project stay on track and handle unexpected issues effectively.

3. Engineering: Design, Coding, and Testing

This phase involved:

- Designing database schemas and UI wireframes.
- Developing frontend (React.js), backend (Node.js/Express), and connecting them with PostgreSQL.

- Implementing authentication and role-based access.
- Conducting unit and integration testing to ensure system stability.

4. Evaluation: Continuous Feedback and Iteration

We gathered feedback through peer reviews and user testing. Based on input, we improved usability, search performance, and UI responsiveness. Iterative development helped refine features and fix issues efficiently before final deployment.

5.2 Feasibility Study

Feasibility analysis is essential to assess whether a project can be executed successfully in terms of technical implementation, economic viability, legal compliance, and user adoption. The "Local Service Provider" has been evaluated based on the following dimensions:

- **Technical**

The project uses widely accepted and mature technologies such as React.js, Node.js, Express.js, and PostgreSQL. The system architecture is designed to be modular, which ensures smooth integration of additional modules in the future (like chat, payments, etc.). Hosting, scalability, and data security are manageable using existing cloud infrastructure and development tools.

- **Economic**

As a student project, there is no substantial cost involved apart from basic development resources. The system runs on free and open-source tools (Node, PostgreSQL, React), making the development and deployment cost-effective. Hosting on free or educational-tier platforms further reduces costs.

- **Operational**

The system is user-centric and designed with simplicity in mind. Its intuitive interface ensures that users can access services and perform bookings without requiring training. Admin functionality is straightforward, allowing smooth management.

- **Legal**

The platform does not collect or store sensitive payment data. All user information is handled securely, complying with basic data protection principles. Any future scaling to real-world deployment would need privacy policies and terms of use.

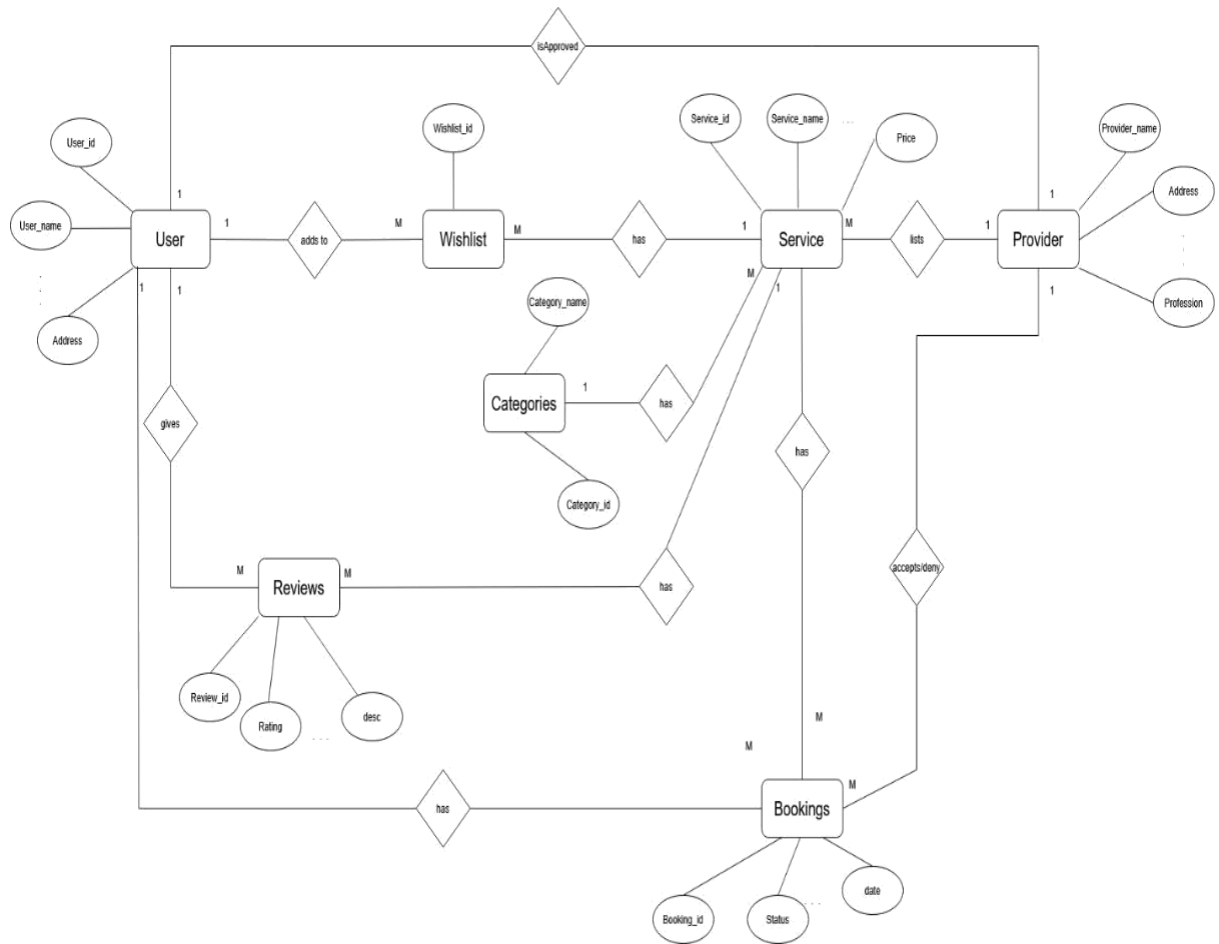


fig 5.1 Entity Relationship Diagram

- **Behavioral**

The system promotes a culture of service reliability and community trust. Feedback and review mechanisms encourage honest transactions and user accountability.

5.3 ER Diagram

The Entity-Relationship (ER) Diagram represents the logical structure of the database. It outlines entities, their attributes, and the relationships among them in the "**Local Service Provider**" system.

Key Entities:

- **User:** Represents general users of the platform. A user can book services, leave reviews, and maintain a wishlist of services they are interested in. A user may also apply to become a service provider.
- **Provider:** Represents users who have been approved to offer services on the platform.
- Providers can list services under various categories, manage their listed services, handle bookings and receive reviews from customers.
- **Service:** Contains details about services offered by providers, including the title, description, category, features, and pricing. Each service can be booked multiple times and reviewed by multiple users.
- **Wishlist:** Represents a list of services that a user is interested in but has not booked yet. Each user can maintain their own personalized wishlist of favorite services.
- **Booking:** Represents a service booking made by a user. It includes information about the user, the service booked, the provider, booking status, and other relevant details. Bookings are confirmed directly between users and providers.
- **Category:** Represents different types of service categories such as Home Cleaning, Plumbing, Electricians, etc. Each service must be associated with a specific category.
- **Review:** Represents feedback submitted by users for services they have booked. Reviews are linked to both the user who submitted them and the service being reviewed, helping other users make informed decision

5.4 Relational Database Design

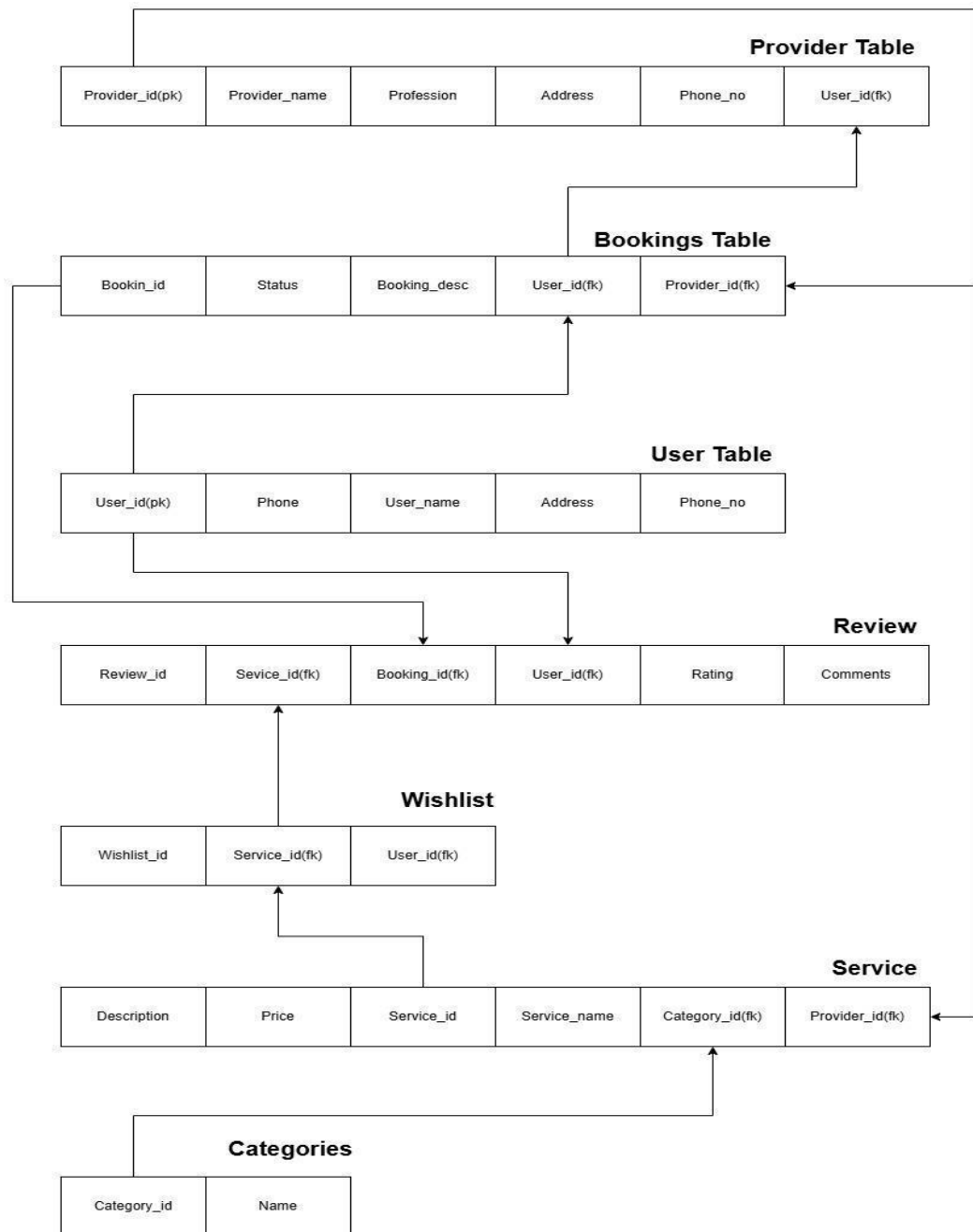


Fig 5.2 Relational Database Management System

The relational database design for the **Local Service Provider** follows a normalized schema that ensures efficient data storage, easy retrieval, and proper enforcement of relationships between different entities. The database is designed using the **PostgreSQL** relational database management system.

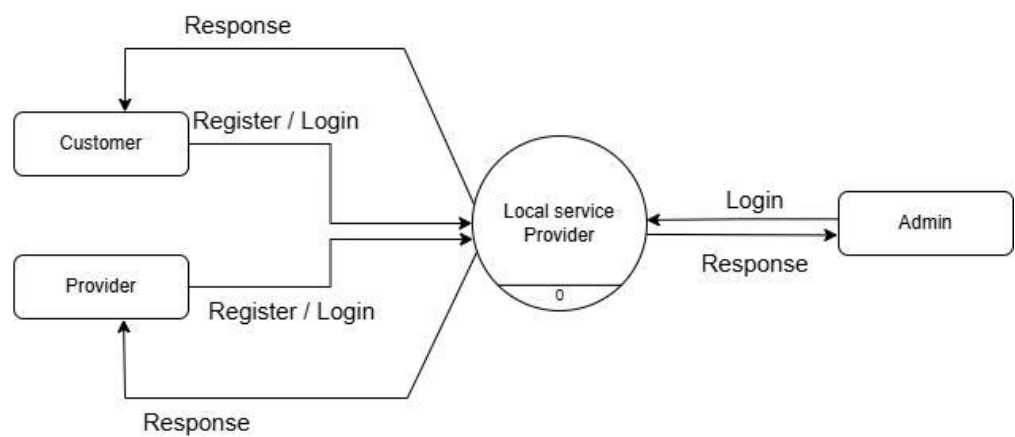
5.5 Data Flow Diagrams

Data Flow Diagrams (DFD) illustrate how data moves through the system. This includes external entities, processes, data stores, and data flows.

Level 0 DFD (Context Diagram)

Represents the system as a single process interacting with users (Service Seekers and Providers) and the Admin.

Level 0 DFD

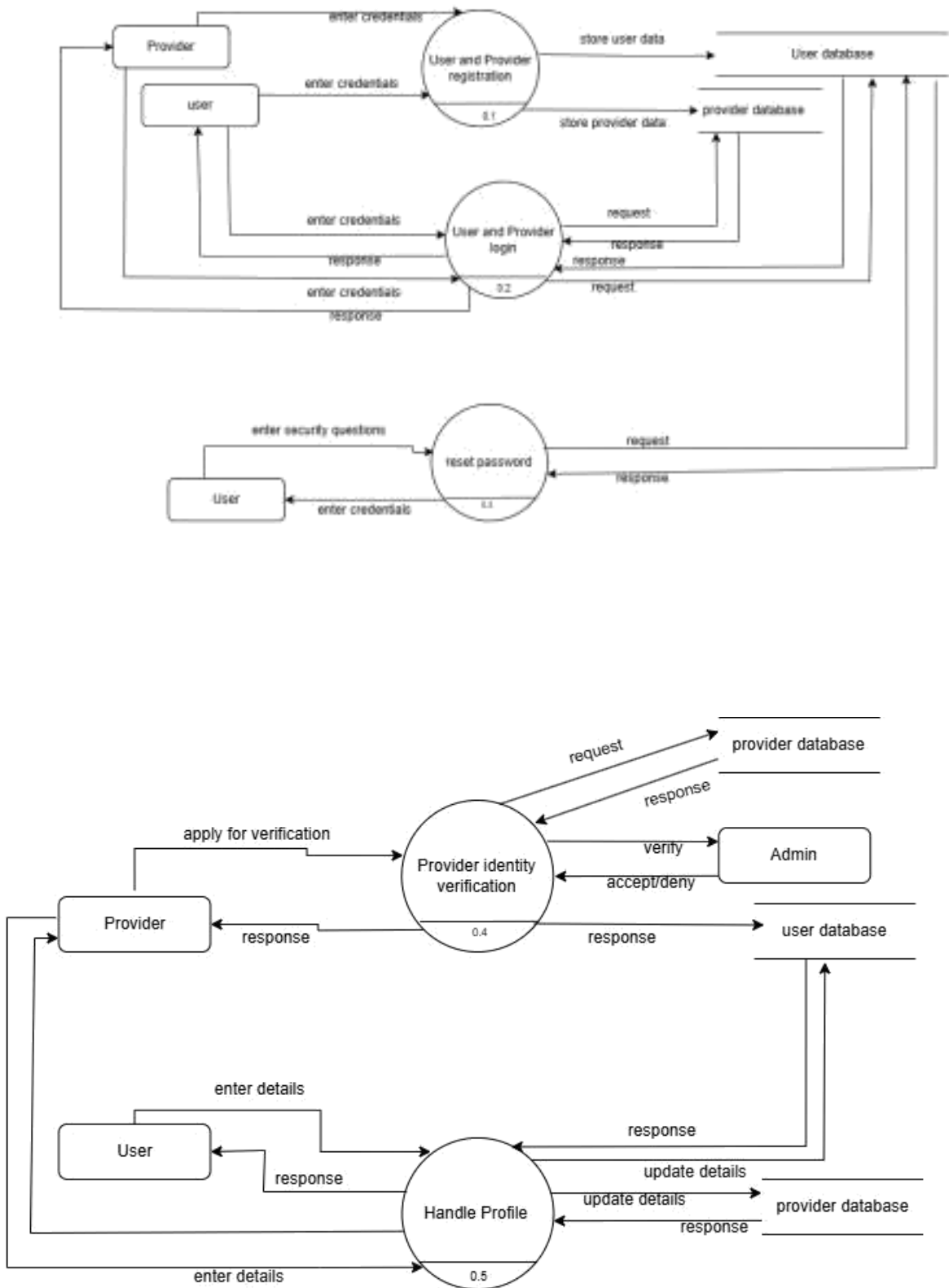


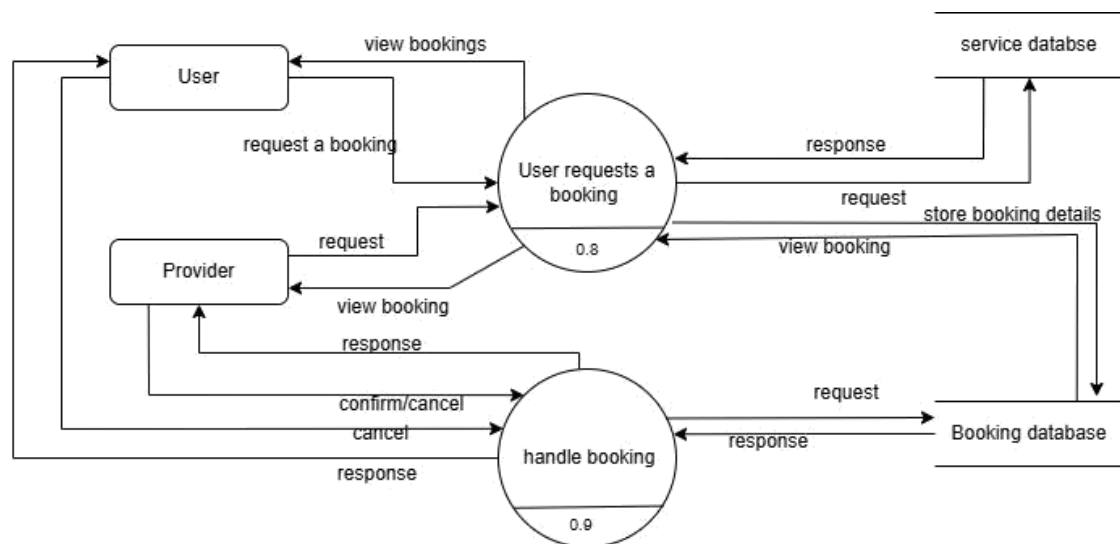
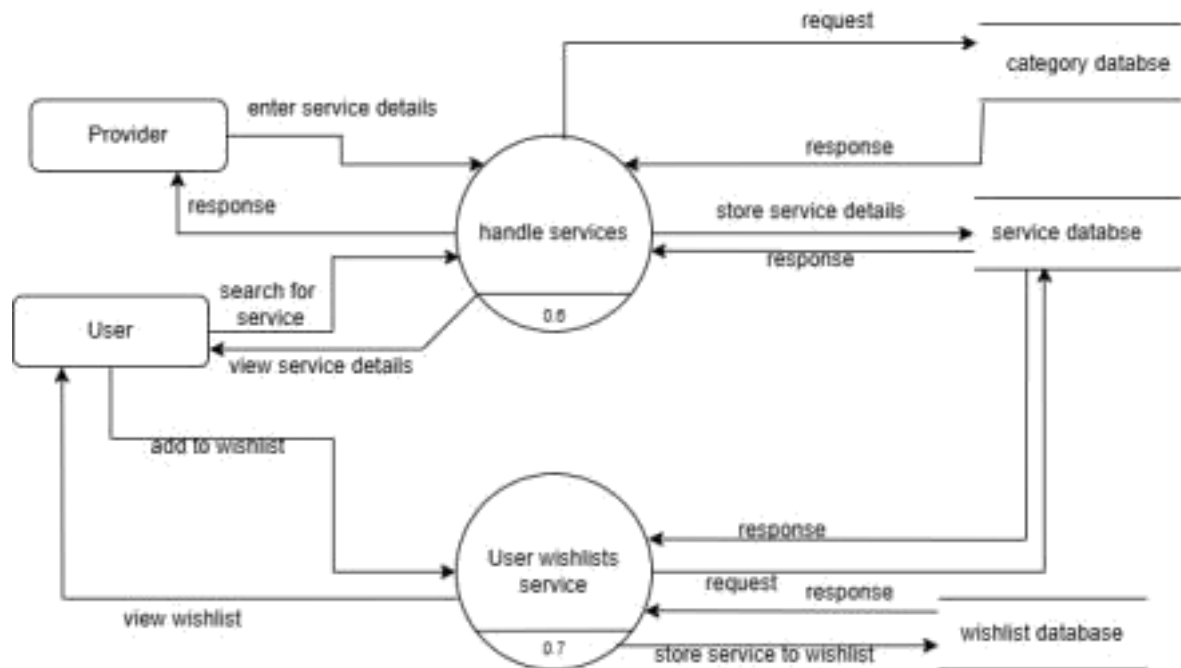
Level 1 DFD

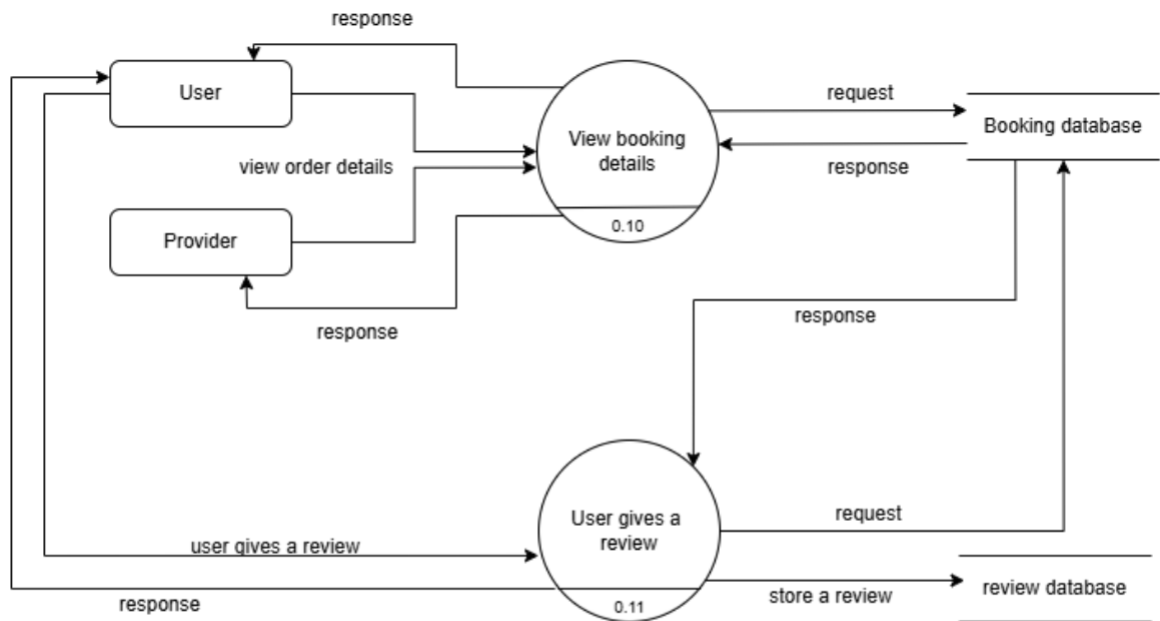
Breaks down the main process into sub-processes:

- User Registration/Login
- Service Browsing and Booking
- Provider Service Management
- Admin Panel Management

Level 1 DFD







Level 2 DFD

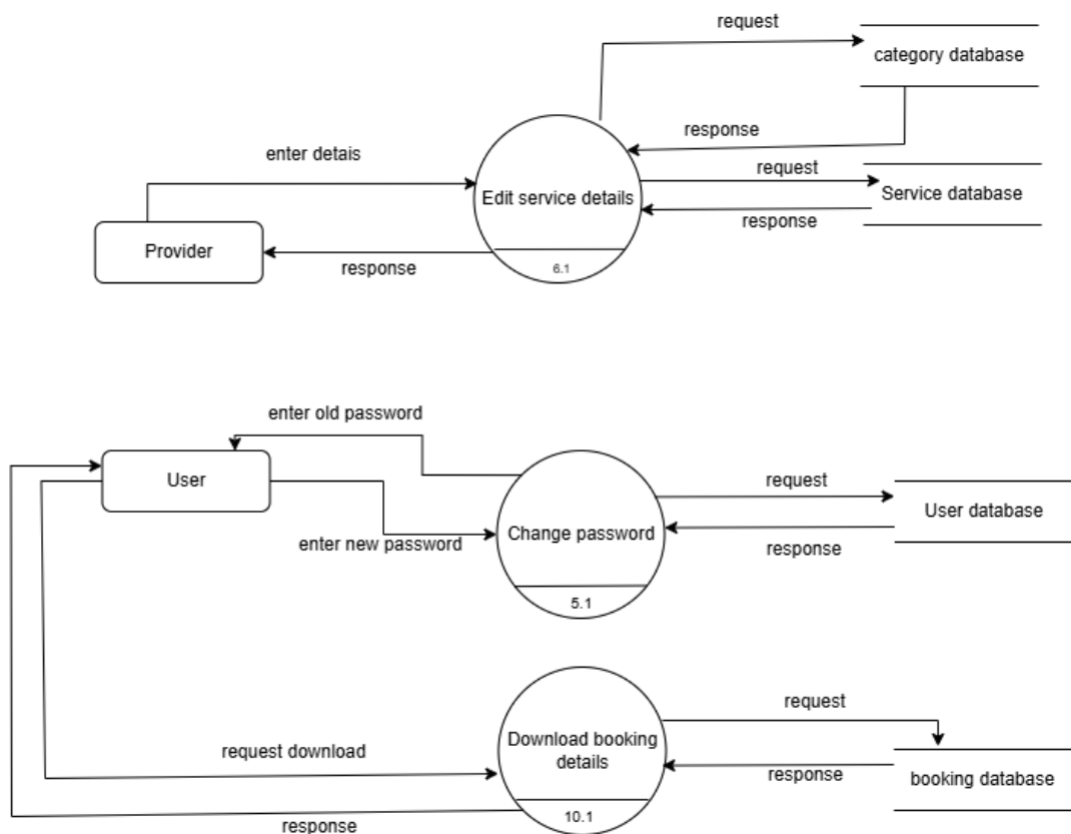


Fig 5.3 Data flow diagram

5.6 Database Design

In the "Local Service Provider Platform," we use **PostgreSQL** Server technology for storing and managing data.

According to the system requirements, functional specifications, and the type of information each entity must hold, the following database tables have been created. The final design of the main tables is described below:

Database Tables Used:

- **Users Table (users)**
Stores general information about all registered users including customers and providers.
- **Providers Table (providers)**
Contains additional provider-specific information for users who have registered as service providers.
- **Services Table (services)**
Contains the details of services offered by providers, including titles, descriptions, categories, prices, and service features.
- **Wishlists Table (wishlists)**
Manages the services that users have added to their wishlist for easy access later.
- **Bookings Table (bookings)**
Stores customer bookings of services, including booking status, booking time, and customer details.
- **Categories Table (categories)**
Defines various service categories (e.g., Plumbing, Gardening, Electrical) under which services are organized.
- **Reviews Table (reviews)**
Stores customer feedback and ratings for services after successful bookings.

5.7 Data Dictionary

USERS

table_name	column_name	data_type	is_nullable	column_default
Users	id	integer	NO	nextval('\"Users_id_seq\"::regclass)
Users	username	character varying	NO	
Users	fullName	character varying	YES	
Users	email	character varying	NO	
Users	password	character varying	NO	
Users	image	character varying	YES	
Users	city	character varying	YES	
Users	address	character varying	NO	
Users	pincode	integer	YES	
Users	phone	bigint	NO	
Users	desc2	text	YES	
Users	isSeller	boolean	YES	false
Users	appliedForProvider	boolean	YES	false
Users	approvedByAdmin	boolean	YES	false
Users	role	string	YES	'user'::\"enum_Users_role\"
Users	securityQA	jsonb	NO	
Users	createdAt	timestamp with time zone	NO	
Users	updatedAt	timestamp with time zone	NO	

INDEXES

table_name	index_name	column_name	is_unique
Users	Users_email_key	email	True
Users	Users_pkey	id	True
Users	Users_username_key	username	True

PROVIDERS

table_name	column_name	data_type	is_nullable	column_default
Providers	id	integer	NO	nextval('Providers_id_seq'::regclass)
Providers	userId	integer	NO	
Providers	verify	character varying	NO	
Providers	desc	text	YES	
Providers	serviceLevel	character varying	YES	
Providers	experience	character varying	YES	
Providers	profession	character varying	YES	
Providers	serviceHours	character varying	YES	
Providers	providerName	character varying	NO	
Providers	providerEmail	character varying	NO	
Providers	providerAddress	character varying	NO	
Providers	providerPhone	bigint	NO	
Providers	dob	timestamp with time zone	NO	

Providers	createdAt	timestamp with time zone	NO	
Providers	updatedAt	timestamp with time zone	NO	

INDEXES

table_name	index_name	column_name	is_unique
Providers	Providers_pkey	id	True
Providers	Providers_userId_key	userId	True

SERVICES

table_name	column_name	data_type	is_nullable	column_default
Services	id	integer	NO	nextval('"Services_id_seq"::regclass)
Services	providerId	integer	NO	
Services	title	character varying	NO	
Services	desc	text	NO	
Services	totalStars	double precision	YES	'0'::double precision
Services	starNumber	integer	YES	0
Services	cat	character varying	NO	
Services	categoryId	integer	YES	
Services	price	numeric	NO	
Services	cover	character varying	NO	
Services	images	ARRAY	YES	([]::character varying[])::character varying(255)[]

Services	shortTitle	character varying	YES	
Services	shortDesc	text	YES	
Services	avgServiceTime	character varying	YES	
Services	revisionNumber	character varying	YES	
Services	features	ARRAY	YES	([]::character varying[])::character varying(255)[]
Services	locationA	ARRAY	YES	([]::character varying[])::character varying(255)[]
Services	locationB	ARRAY	YES	([]::character varying[])::character varying(255)[]
Services	sales	integer	YES	0
Services	isUnlisted	boolean	YES	false
Services	isDeleted	boolean	YES	false
Services	availability	jsonb	YES	'[]'::jsonb
Services	search_vector	tsvector	YES	
Services	createdAt	timestamp with time zone	NO	
Services	updatedAt	timestamp with time zone	NO	

INDEXES

table_name	index_name	column_name	is_unique
Services	Services_pkey	id	True

CATEGORIES

table_name	column_name	data_type	is_nullable	column_default
Categories	id	integer	NO	nextval('Categories_id_seq'::regclass)
Categories	name	character varying	NO	
Categories	description	text	YES	
Categories	cover	character varying	YES	
Categories	isActive	boolean	YES	true
Categories	createdAt	timestamp with time zone	NO	
Categories	updatedAt	timestamp with time zone	NO	

INDEXES

table_name	index_name	column_name	is_unique
Categories	Categories_name_key	name	True
Categories	Categories_pkey	id	True

WISHLISTS

table_name	column_name	data_type	is_nullable	column_default
Wishlists	id	integer	NO	nextval('Wishlists_id_seq'::regclass)
Wishlists	userId	integer	NO	
Wishlists	serviceId	integer	NO	

INDEXES

table_name	index_name	column_name	is_unique
Wishlists	Wishlists_pkey	id	True
Wishlists	Wishlists_userId_serviceId_key	userId	True
Wishlists	Wishlists_userId_serviceId_key	serviceId	True

BOOKINGS

table_name	column_name	data_type	is_nullable	column_default
Bookings	id	integer	NO	nextval('"Bookings_id_seq"'::regclass)
Bookings	serviceId	integer	NO	
Bookings	img	character varying	YES	
Bookings	title	character varying	NO	
Bookings	email	character varying	YES	
Bookings	price	numeric	NO	
Bookings	preferredDate	date	NO	
Bookings	preferredTime	character varying	NO	
Bookings	notes	text	YES	
Bookings	providerId	integer	NO	
Bookings	buyerId	integer	NO	
Bookings	status	string	NO	'Pending'::"enum_Bookings_status"
Bookings	createdAt	timestamp with time zone	NO	
Bookings	updatedAt	timestamp with time zone	NO	

INDEXES

table_name	index_name	column_name	is_unique
Bookings	Bookings_pkey	id	True

REVIEWS

table_name	column_name	data_type	is_nullable	column_default
Reviews	serviceId	integer	NO	
Reviews	userId	integer	NO	
Reviews	bookingId	integer	NO	
Reviews	star	integer	NO	
Reviews	desc	text	NO	
Reviews	createdAt	timestamp with time zone	NO	
Reviews	updatedAt	timestamp with time zone	NO	

INDEXES

table_name	index_name	column_name	is_unique
Reviews	Reviews_pkey	id	True

Figure 5.4 Data Dictionary

Chapter-VI

Project Structure

6.1 Modules

The project is divided into several functional modules to ensure clarity, scalability, and ease of development.

Key Modules:

- **Authentication Module:** Handles user registration, login, and session management.
- **User Module:** Enables users to update profiles, view services, and request services.
- **Service Provider Module:** Allows providers to manage their services, view requests, and update statuses.
- **Admin Module:** Admin can manage users, providers, and service categories.
- **Service Management Module:** Manages all service listings, categories, and bookings.
- **Location Module:** Stores and processes city and area data for location-based filtering.

6.2 Data Structures and APIs

Frontend (React):

- **Data Structures:** State management via React Hooks and Context API.
- **Data Format:** JSON used for all data exchanges between client and server.
- **Data Structures:** JavaScript objects; Sequelize for interacting with PostgreSQL.
- **APIs:** RESTful endpoints for CRUD operations.

Example API Endpoints:

Method	Endpoint	Description
POST	/api/auth/register	Register new user
POST	/api/auth/login	Login user
GET	/api/services	Get all services
POST	/api/requests	Create service request
PUT	/api/requests/:id	Update request status

6.3 Process Logic / Workflows

User Workflow:

1. Register/Login.
2. Search for services by category or location.
3. Send a service request.
4. View request status.

Service Provider Workflow:

1. Register/Login.
2. Set up service details (type, location, availability).
3. View and accept/reject requests.
4. Update request status once completed.

Admin Workflow:

1. Login to dashboard.
2. Monitor user/provider activity.
3. Add/update/remove categories and monitor service usage.

6.4 Screens and Their Functions

Screen Name	Functionality
Home Page	Overview of the platform and featured services
Login/Register Page	Authentication for users and providers
User Dashboard	Profile management, service browsing, and request tracking
Service Listing Page	Filter/search services by category and location
Request Form Page	Form to submit service requests
Provider Dashboard	Manage services, view requests, update status
Admin Panel	Manage categories, users, and monitor activity

Chapter-VII

Testing

7.1 Unit Testing

Unit testing was conducted to validate individual components of both the frontend and backend systems. Each function, module, and model was tested in isolation to ensure that they performed as expected. Critical areas such as service listing, booking creation, wishlist operations, and review submission were tested through unit tests. Mock data was used to simulate various user actions and edge cases.

Purpose: To ensure that each part of the system works correctly in isolation without relying on external dependencies.

Unit Case-1

Name of Test	Login Test
Item being tested	Test for valid user login credentials
Sample Input	Enter wrong username or password
Expected Output	Username or password is incorrect
Actual Output	Same as expected output
Result	Successful

Unit Case-2

Name of Test	New User Registration Test
Item being tested	Test for valid user registration data
Sample Input	Enter incomplete or invalid user details
Expected Output	Error: Correct user details required
Actual Output	Same as expected output
Result	Successful

Unit Case-3

Name of Test	Service Creation Test
Item being tested	Test for Provider's service creation
Sample Input	Submit a service form with missing required fields
Expected Output	Error: Fill required fields
Actual Output	Same as expected output
Result	Successful

Unit Case-4

Name of Test	Booking Test
Item being tested	Test for successful service booking
Sample Input	Book a service by selecting a valid date and details
Expected Output	Booking confirmed successfully
Actual Output	Same as expected output
Result	Successful

Unit Case-5

Name of Test	Review submitting Test
Item being tested	Test adding a review to a service
Sample Input	Click on "Submit review" for a service
Expected Output	Review added to service
Actual Output	Same as expected output
Result	Successful

7.2 Functional Testing

Feature	Test performed	Purpose	Result
User Registration	Tested registration process for customers and providers.	To ensure users can successfully create accounts.	Pass
Service Creation	Verified that providers can create and manage service listings.	To confirm providers can list services as intended.	Pass
Booking a Service	Simulated booking flow from selecting a service to confirmation.	To validate the complete booking workflow.	Pass
Wishlist Management	Checked adding/removing services to/from wishlist.	To ensure users can manage their favorite services.	Pass
Leaving Reviews	Tested review submission after service completion.	To verify that only eligible users can leave feedback.	Pass
Admin Approvals	Verified admin workflow for approving or rejecting provider applications.	To ensure provider vetting process works as designed.	Pass

Functional testing confirmed that the system workflows were correct and consistent.

7.3 Tools Used for Testing

The following tools were utilized to perform testing throughout the development lifecycle:

- **Postman:** For testing REST API endpoints and verifying backend functionality.
- **React Testing Library:** For simulating user interactions and testing React components.
- **PgAdmin 4:** For verifying database operations and query results during backend testing.
- **Browser Developer Tools (Chrome DevTools):** For manual UI testing, checking responsiveness, console errors, and network activities.
- **Manual Testing:** For exploratory testing to catch UI glitches, usability issues, and functionality bugs.

Purpose: To maintain a high-quality, bug-free, and user-friendly application throughout the development cycle.

Chapter-VIII

Limitations and Future Enhancements

8.1 System Limitations

While the current system meets the core requirements and provides a fully functional platform for service booking, it does have some limitations:

No Online Payments & In-App Communication:

The platform lacks integrated payment gateways and in-app chat, requiring users to handle transactions and communication externally.

Limited Real-Time Functionality:

Booking status and availability rely on manual refresh rather than real-time updates via technologies like WebSockets.

Basic Admin & Security Features:

The admin panel and security setup are minimal, missing advanced controls, two-factor authentication, and activity monitoring.

Scalability & Accessibility Constraints:

The system hasn't been tested under high traffic, supports only English, and may not scale well without optimization.

8.2 Future Enhancements

To make the platform more robust, user-friendly, and scalable, the following enhancements are planned for future development:

Online Payments & In-App Communication:

Integrate a secure payment gateway and build an in-app chat system to streamline transactions and communication between users and providers.

Real-Time & Mobile Access:

Implement real-time notifications using WebSockets or push services, and develop mobile apps for Android and iOS to improve accessibility and user engagement.

Advanced Admin & Security Features:

Upgrade the admin dashboard with analytics, activity monitoring, and revenue tracking, while enhancing platform security with 2FA, encryption, and activity logs.

Scalability, Language Support & AI Integration:

Optimize system performance for high traffic, add multi-language support for broader accessibility, and introduce AI-driven recommendations for personalized user experiences.

Vision:

To evolve the platform into a comprehensive, scalable, and intelligent service marketplace, providing seamless experiences for both users and providers.

Conclusion

The development of the “**Local Service Provider**” platform successfully delivers a user-friendly solution for connecting customers with a wide range of service providers. By implementing key features such as service listings, bookings, wishlists, reviews, and admin management, the project meets its core objectives. Through structured testing and continuous improvements, a stable and functional system was achieved. While there are areas for future growth, the project lays a strong foundation for expanding into a fully scalable, real-time, and feature-rich platform. This project has also provided valuable learning experiences in full-stack development, database management, testing, and project management.

References

Book :

- R. Elmasri, S.B. Navathe, Fundamentals of Database Systems 6th Edition, Pearson Education, 2010.
- Silberschatz, H.F. Korth, S. Sudarshan, Database System Concepts 6th Edition, McGraw Hill, 2010.

Portals

- www.geekforgeeks.com
- <https://nodejs.org/en>
- <https://expressjs.com>
- <https://vite.dev>
- www.youtube.com
- ☐ <https://react.dev>
- ☐ <https://www.postgresql.org>
- ☐ <https://www.npmjs.com>
- ☐ <https://yarnpkg.com>
- ☐ <https://www.freecodecamp.org>
- ☐ <https://mui.com>
- ☐ <https://cloudinary.com>
- <https://gitProvider.com>

USER GUIDE

1. Introduction

Welcome to the Local Service Provider Platform. This platform connects customers with local professionals (e.g., plumbers, electricians) for booking services easily and efficiently.

2. Getting Started

2.1. System Requirements

- A modern browser (Chrome, Firefox, Edge)
- Internet connection
- Desktop, laptop, or mobile device

3. User Roles

Role	Description
Customer	Can register, browse services, book services, leave reviews
Provider	Can register, list services, manage orders
Admin	Manages users, approves provider applications, monitors activity

4. Customer Guide

- Click on **Sign Up**
- Fill in your details and complete registration
- Login via the **Login** button after registration

4.2. Browsing Services

- Use the search bar or categories to explore services
- Click on a service to view details

4.3. Booking a Service

- Click on **Book Now**
- Fill the booking form
- Submit and for confirmation
- Download a PDF with booking info

4.4. Managing Wishlist

- Click the ♥ icon to add a service to your wishlist
- View all wishlisted gigs from the **Wishlist** page

4.5. Leaving Reviews

- After booking completion, visit the gig page to leave a review
- Only confirmed orders allow reviews

5. Provider Guide

5.1. Becoming a Provider

- Register normally
- Toggle "Become a Provider" while registering
- Fill out necessary provider related details
- Wait for admin approval

5.2. Creating Services

- Click on the **Profile Icon**
- Go to **My Services**
- Click **Add New Service**
- Fill in service details and submit

5.3. Managing Bookings

- View and accept/reject bookings

5.4. Edit / Unlist Services

- Use **Edit** to update info
- Use **Unlist** to temporarily remove a service from public view
- Use **Delete** to remove a service from **My services** and marketplace

6. Admin Guide

6.1. Admin Login

- Log in using admin credentials

6.2. User & Provider Management

- View users and providers
- Approve or reject provider requests
- Revoke provider access if needed

6.3. Dashboard Overview

- View total users, bookings, providers
- Access data and reports on system usage

7. Support

For help, contact the Admin or refer to the FAQ section on the website.

Screenshots

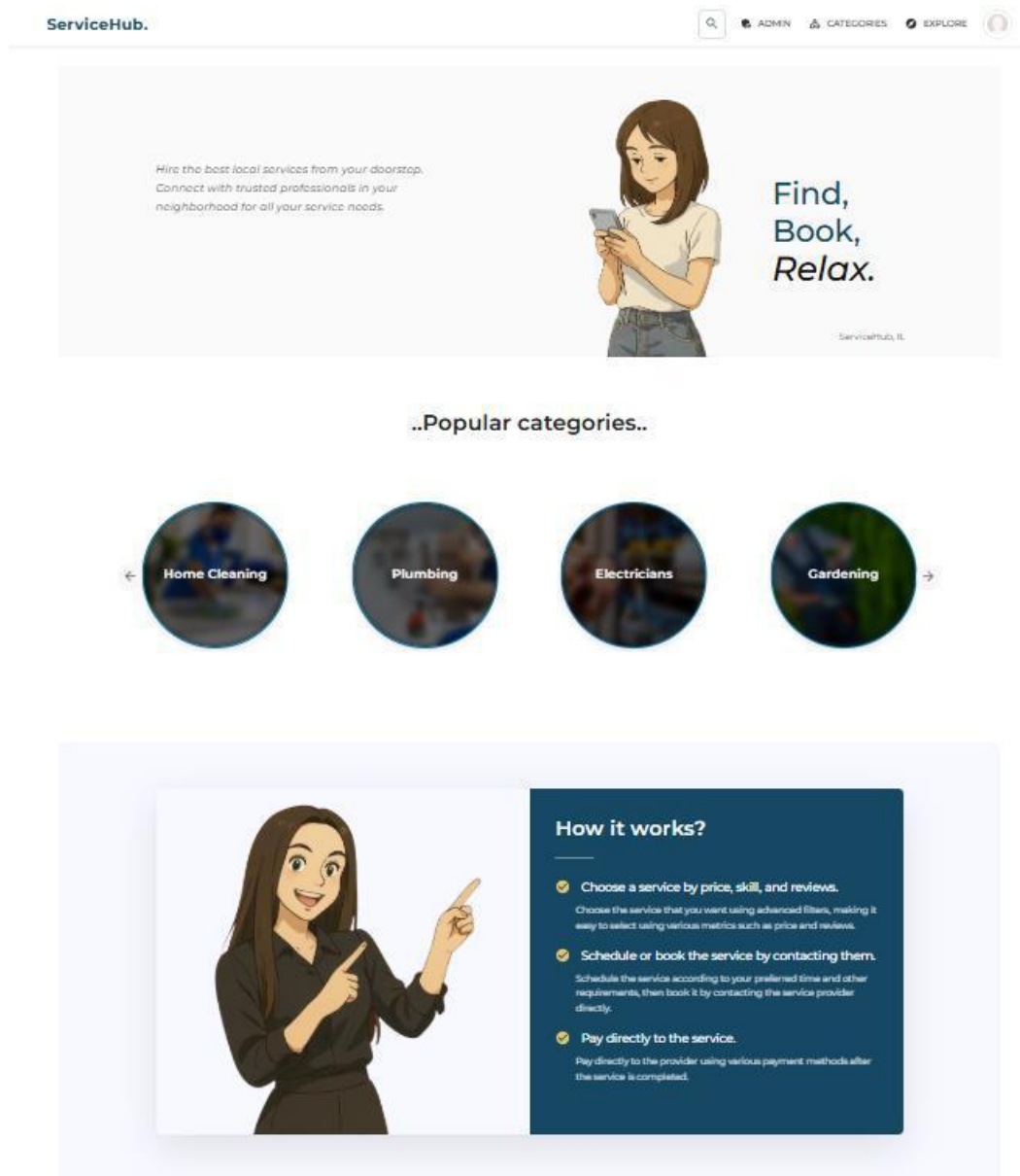


Fig 1 Home page

Create Your Account

Join our community and unlock all features. Fill in your details below to get started. Fields marked * are required for successful registration

1

2

3

Basic Information

Additional Details

Finalize

Username* Don	Phone Number* 7879874544
Full Name Don Das	Address* Assam, North Lakhimpur
Email* david@gmail.com	Pincode 787001
Password* 	

CONTINUE →

Fig 2 Register page

Create Your Account

Join our community and unlock all features. Fill in your details below to get started. Fields marked * are required for successful registration

1

2

3

Basic InformationAdditional DetailsFinalize

City

About you

🏠

Become a Provider

?

No

Switch on to apply as a service provider and earn by offering your expertise.

🔒

Security Questions

Please select at least 3 security questions and provide answers to help recover your account if you forget your password.

Security Question 1*

Answer 1*

Security Question 2*

Answer 2*

Security Question 3*

Answer 3*

👤

Profile Picture

CHOOSE FILE

UPLOAD

BACK

→

Fig 3 Register page

Become a Provider

 Yes

Switch on to apply as a service provider and earn by offering your expertise.

Fig 4 - Become a Provider

Create Your Account

Join our community and unlock all features. Fill in your details below to get started. Fields marked * are required for successful registration.

✓

✓

3

Basic Information

Additional Details

Provider Information

Business Name*

Seller1

Business Email*

seller1@gmail.com

Business Address*

adad

Business Phone*

8888888888

Date of Birth*

02-04-2025

Service Level

Individual

Service Hours

9 AM - 10 AM Monday-Friday

Profession

Plumber

Experience

5 years

About your service

very good

Identity Verification*



SELECT ID

UPLOAD

Please upload a valid ID proof. We accept government-issued photo ID.

Almost Done!

Please review your information before submitting. Once submitted, you'll be directed to the login page.

BACK

COMPLETE REGISTRATION

Fig 5 Register page (as a provider)

Create Your Account

Join our community and unlock all features. Fill in your details below to get started. Fields marked * are required for successful registration

✓

✓

3

Basic InformationAdditional DetailsFinalize

Almost Done!

Please review your information before submitting. Once submitted, you'll be directed to the login page.

BACKCOMPLETE REGISTRATION

Welcome Back

Enter your credentials to access your account

Username*

 Admin

Password*

→ Sign In

OR

Don't have an account? [Register here](#)

Forget account password? [Reset Password](#)

Fig 6 Login page (as admin)

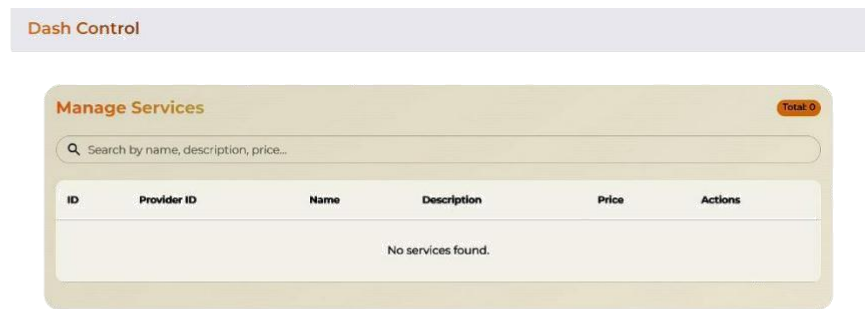
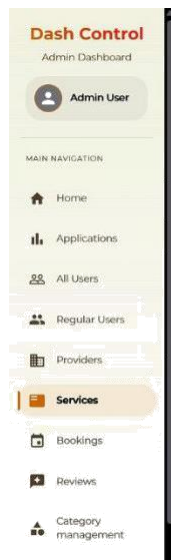


Fig 7 Admin dashboard

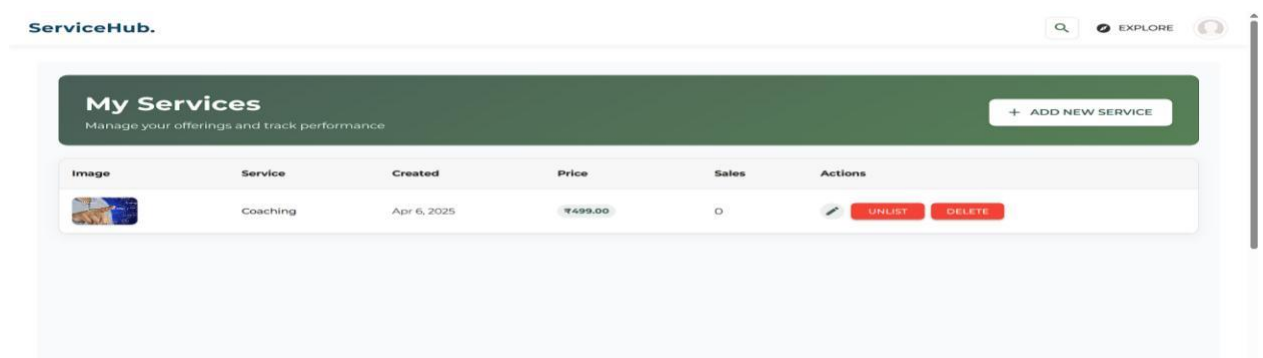


Fig 8 List and manage services

Create Your New Service

Basic Information

Service Name*

Description*

Category

Service Images

Upload Cover Image

Upload Gallery Images (Max 4)

Upload Images

Maximum Service: 200KB per image

Additional service details

Additional Description

Availability Schedule

No availability dates added yet. Add your first date below.

Add Availability Date

Service Features

e.g. 10000 Cleaning, 1000000, 1000

No features added yet

Service Time

e.g. 3 hours

Price

₹

Service Area

Define where your service is available to reach the right customers.

Pincode

Add service areas by pincode

e.g. 700031

No pincodes added yet

City

Add cities where your service is available

e.g. Guwahati

No cities added yet

Create Service

Fig 9 Create services

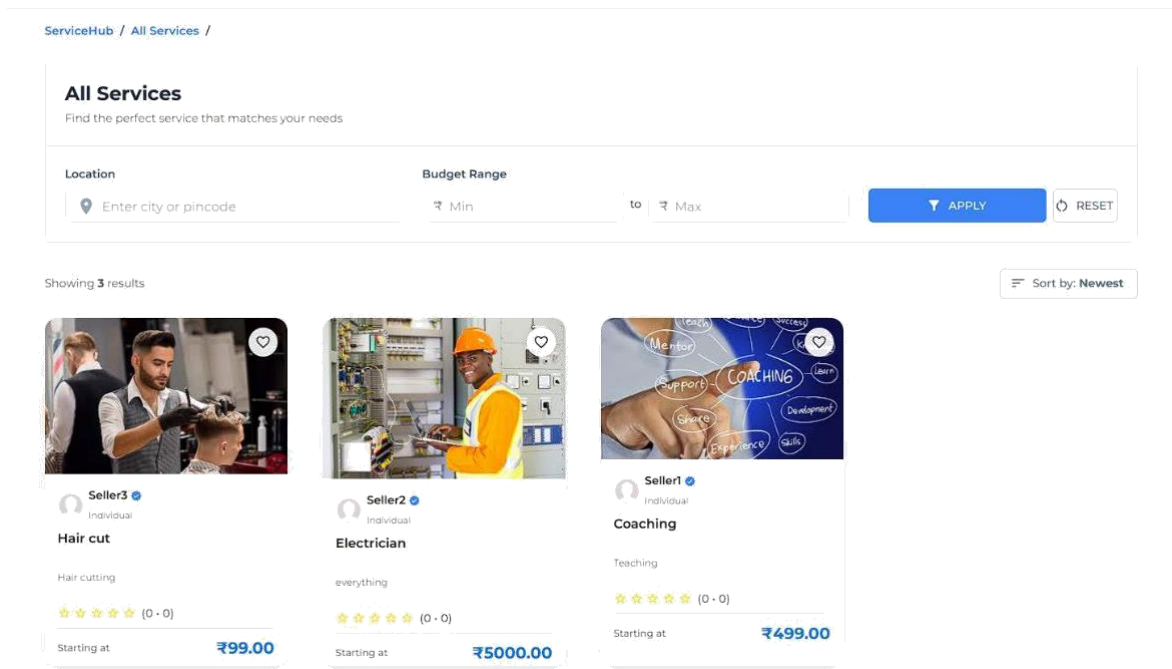


Fig 10 Service marketplace

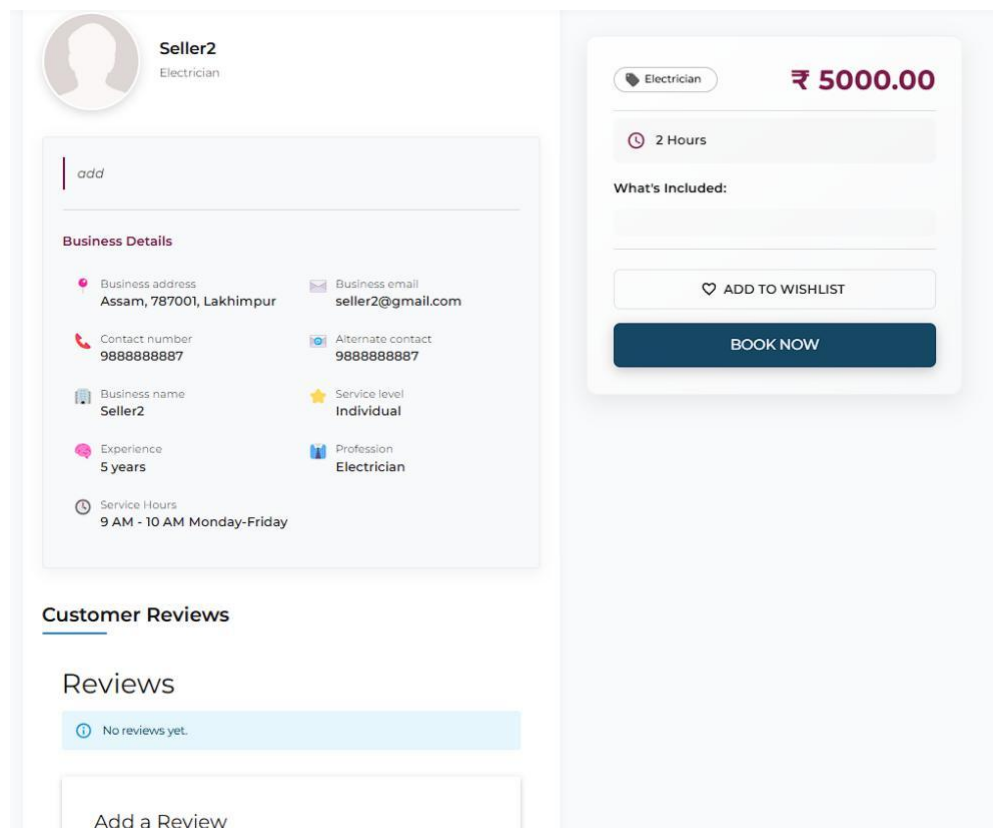


Fig 11 Service details

Book Your Service

Electrician

Personal Information

Name*

Don

Phone*

7879874544

Email*

david@gmail.com

Address*

Assam, North Lakhimpur

Booking Details

Preferred Date*

 17-04-2025

Preferred Time*



No available slots for this date. Please try another date.

Additional Notes

Please let us know any specific requirements or instructions...



Confirm Booking

Fig 12 Booking a service

My Bookings

Track your bookings and appointments

Filters

AS BUYER

Status

All

Service	Price	Date	Status	Actions
 Electrician	₹5000.00	Apr 6, 2025	Pending	✕ ↻

<

1

>

Fig 13 Manage bookings

ServiceHub.

Q

CATEGORIES

EXPLORE

Order Summary

Service Details



coaching

₹1001.00

Expired

Preferred Date: 2025-04-16

Preferred Time: 06:24

Notes: No additional notes provided

Customer Details



suraj

suraj@gmail.com

Name:

Phone: 7777799999

Address: lakhimpur, -

Provider Details



mitu

pcoaching@gmail.com

Business name: Provider Coaching

Business Phone: 8471869547

Business Address: k.B Road, Lakhimpur, -

Additional Provider Information

Profession: Coaching

Experience: 20 years years

Service Hours: 9 AM to 6 PM

Service Level: Individual

Service description:

DOWNLOAD ORDER DETAILS AS PDF

Fig 14 View and download booking details

61